Maastricht University

Department of Advanced Computing Science

Algorithmic Design

Week 3:
Dynamic Programming
(or more fun with recurrances)

David Mestel and **Steven Chaplick**

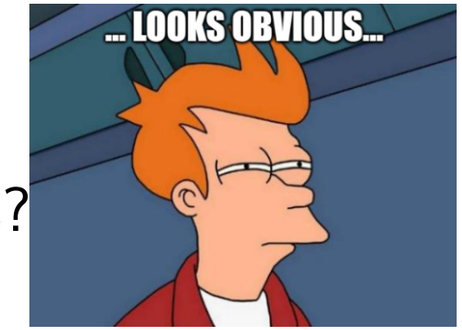
2025-2026 Period 5

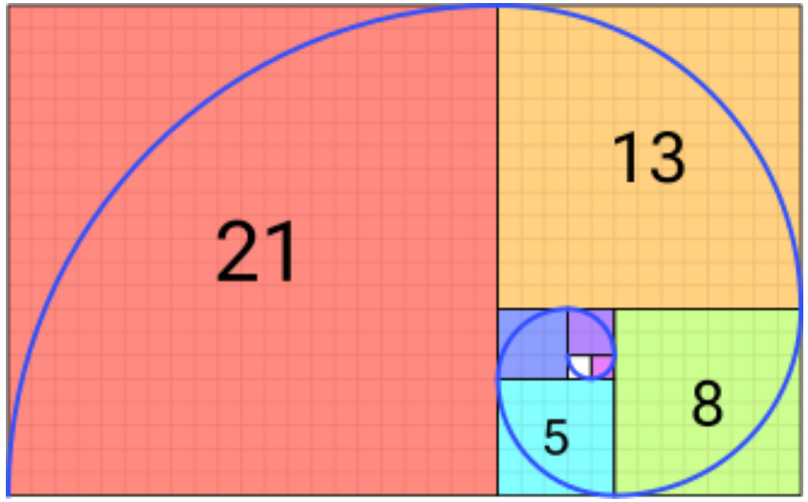
Warm-up : computing Fibonacci Numbers (part 1)

$F_n = F_{n-1} + F_{n-2}$ where

$F_0 = 0, F_1 = 1$

But, how to compute F_n for some given n?





Fibonacci approx. of Golden Sprial fractal.
source: wikipedia

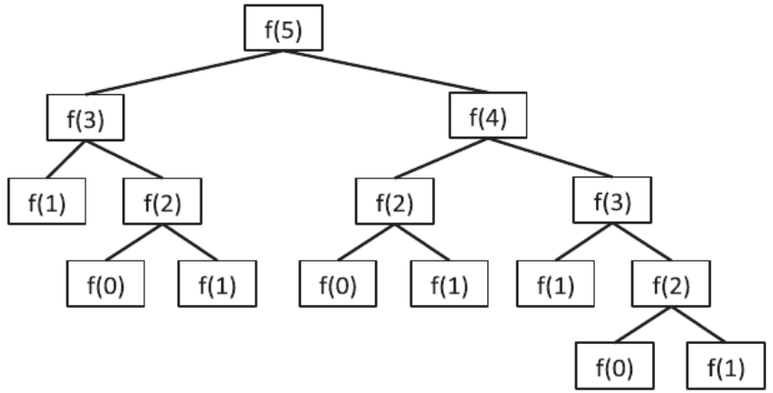
```
function F(int n) \\assumes n >= 0
   if n = 0 or n = 1 { return n }
   else { return F(n-1) + F(n-2) }
end function
```

Runtime $T(n) = T(n-1) + T(n-2) + O(1)$

$(n > 2)$ $\geq 2 \cdot T(n-2) + O(1)$

$\geq 2^{\lfloor \frac{n}{2} \rfloor} + O(1)$

Brutally slow ...



Warm-up : computing Fibonacci Numbers (part 2)

$F_n = F_{n-1} + F_{n-2}$ where

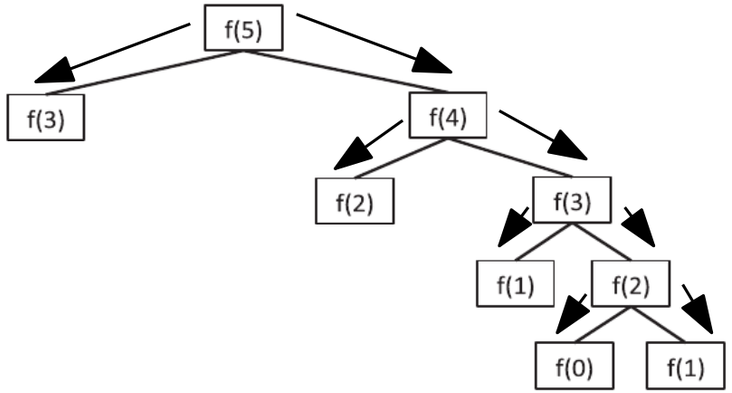
$F_0 = 0, F_1 = 1$

But, how to compute F_n **faster** for some given n?

Memoization: save the result of recursive calls to avoid repeated work

```
function F(int n) \\assumes n >= 2
   memoF := int(n+1); initialize all as -1;
   memoF[0] := 0;    memoF[1] := 1;
   return recF(memoF, n)
end function
```

```
function recF(memoF, int i)
   if memoF[i-1] = -1
       memoF[i-1] := recF(memoF, i-1)
   else if memoF[i-2] = -1
       memoF[i-2] := recF(memoF, i-2)
   return memoF[i-1] + memoF[i-2]
end function
```



Runtime:

$$T(n) = T(n-1) + O(1)$$
$$= O(n)$$

Much better :)

Warm-up : computing Fibonacci Numbers (part 3)

$F_n = F_{n-1} + F_{n-2}$ where
$F_0 = 0, F_1 = 1$

But, **I hate recursion**, can I still compute F_n ?
Yes! Fill in the **memoized** entries **bottom-up**.

```
function F(int n) \\assumes n >= 2
   memoF := int(n+1);
   memoF[0] := 0;    memoF[1] := 1;
   for int j from 2 ... n
       memoF[j] := memoF[j-1] + memoF[j-2]
   return memoF[n]
end function
```

Runtime? $O(n)$:)

Here `memoF` is the **table** of our dynamic program.
Base Cases & formula `memoF[j] := memoF[j-1] + memoF[j-2]` $\rightsquigarrow$ **recurrance**

Exercise: `memoF` has $\Theta(n)$ entries $\sim$ memory. Can we compute F_n using $O(1)$ memory?

Ok, so what is Dynamic Programming?

Standard steps to create a dynamic programming algorithm

- Given an optimization problem, find a recursive algorithm that solves it, the **recurrance**.
- Realize that the distinct **subproblems** visited in the recursive algorithm are not too many and can be memoized, the **table**.
- Reformulate with a bottom-up iterative implementation.

Recall that with greedy algorithms we also spoke of subproblems, but there we grew one single solution (to rule them all!).

Now, we dynamically maintain several partial solutions

Problems to Study

Longest Common Subsequence (LCS) [GT 12.5 , CLRS 15.4]

- Measuring the similarity of two strings, e.g., two strands of DNA.

Scheduling Telescope Time [GT 12.3]

- Activity Selection, where activities have **benefits**

Knapsack returns [GT12.6], [CLRS 16.2 (+exercise 35-7)]

- Can we really solve it in polynomial time?

Travelling Salesperson (TSP)

- Finding the fastest way to deliver all packages (visit all important locations) and return home.
- Exponential running times

Longest Common Subsequences

Geneticists are trying to measure the **commonality** between two strands s_1 and s_2 of DNA. The strands are given as strings of characters `A`, `C`, `G`, `T` corresponding to nucleotides.

Idea read left-to-right in both strings and make two **subsequences** that are the same.

```
X: ACCGGTCGAGTGCGCGGAAGCCGGCCGAA
     | ||  || || | ||||||||||||
     G TC  GT CG G AAGCCGGCCGAA
    GTCGT CGGAA GCCG  GC C G AA
    ||||| ||||| ||||  || | | ||
Y:  GTCGTTCGGAATGCCGTTGCTCTGTAA
```

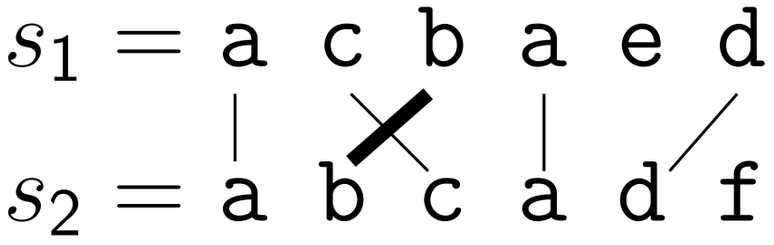
For two strings s and t,

- s is a **subsequence** of $t :=$ delete entries in t to get s

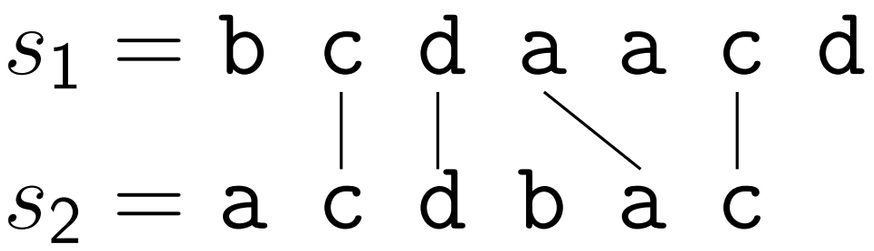
The measure of **commonality** is the length of a longest such subsequence.

Goal: Given two strings s_1 and s_2 compute a longest common subsequnce (LCS) of s_1 and s_2

Build Intuition



acad
abad



cdac

Brute Force: (enumerate all possible feasible solutions and picking the best)

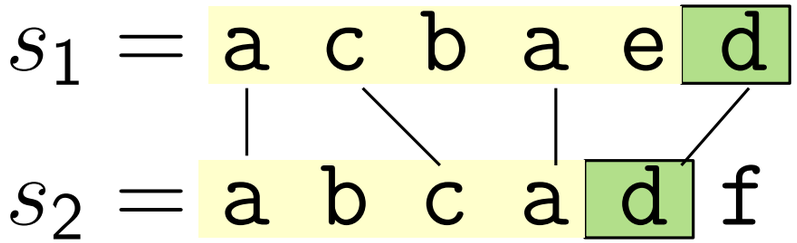
Let $|s_1| = n$, $|s_2| = m$. How many subsequences are there of (i) s_1? 2^n (ii) s_2? 2^m

Compare all pairs of such subsequnces and pick the longest equal pair: $O(2^{n+m} \cdot (n+m))$

Dynamic Programming for LCS

What recursive structure can we find in the subproblems?

- What are the subproblems?
  - Consider substrings s_1' of s_1 and s_2' of s_2.
- Natural Base Cases?
  - Trivial Cases: $|s_1| = 0$. Optimal solution is the empty string, length 0.
  - $|s_1| = 1 \rightsquigarrow$ Opt is s_1 (if s_1 is a character of s_2), or empty string (otherwise).
- Relate solution of a "big" subproblem to solutions to "smaller" subproblems.



drop last char. : $s_1' =$ a c b a e

Use the last char. of $s_1 \Rightarrow$
substring s_2' of s_2 where Opt(s_1',s_2') can be **extended**

Notation

- Let s_1 and s_2 be our strings, where $|s_1| = n$, $|s_2| = m$ and (wlog) $n \leq m$.
- To address single characters, we treat s_1 and s_2 as arrays, e.g., $s_1[1]$ is the first character of s_1, and $s_2[6]$ is the sixth character of s_2.
- Similar for substrings, e.g., $s_1[1...5]$ is the substring of s_1 consisting of the first 5 characters.
- For $s'_1 = s_1[k...i]$ and $s'_2 = s_2[l...j]$, let $L(s'_1, s'_2) :=$ **length** of an LCS of s'_1 and s'_2.

Table

$s_1 =$ a c b a e d

$s_2 =$ a b c a d f

$L(s_1[1...n], s_2[1...m-1]) = 4$

$s_1 =$ a c b a e d

$s_2 =$ a b c a d f

$L(s_1[1...n-1], s_2[1...m-1]) = 3$

$L(s_1[1...n-1], s_2[1...m-2]) = 3$

Notation

- Let s_1 and s_2 be our strings, where $|s_1| = n$, $|s_2| = m$ and (wlog) $n \leq m$.
- To address single characters, we treat s_1 and s_2 as arrays, e.g., $s_1[1]$ is the first character of s_1, and $s_2[6]$ is the sixth character of s_2.
- Similar for substrings, e.g., $s_1[1...5]$ is the substring of s_1 consisting of the first 5 characters.
- For $s_1' = s_1[k...i]$ and $s_2' = s_2[l...j]$, let $L(s_1', s_2') :=$ **length** of an LCS of s_1' and s_2'.

How many substrings to consider for (i) s_1? $O(n^2)$ (ii) s_2? $O(m^2)$

Table

This would make our table L of size $O(n^2m^2)$. Does it really need to be this big?

- enough to consider **prefix** substrings, i.e., $s_1[1...i]$ and $s_2[1...j]$.
  $\rightsquigarrow$ table size $O(nm)$.

So, how to compute the value $L(s_1[1...i], s_2[1...j])$, shorthand $L(i, j)$.

Note: at the end we only really need $L(n, m)$.

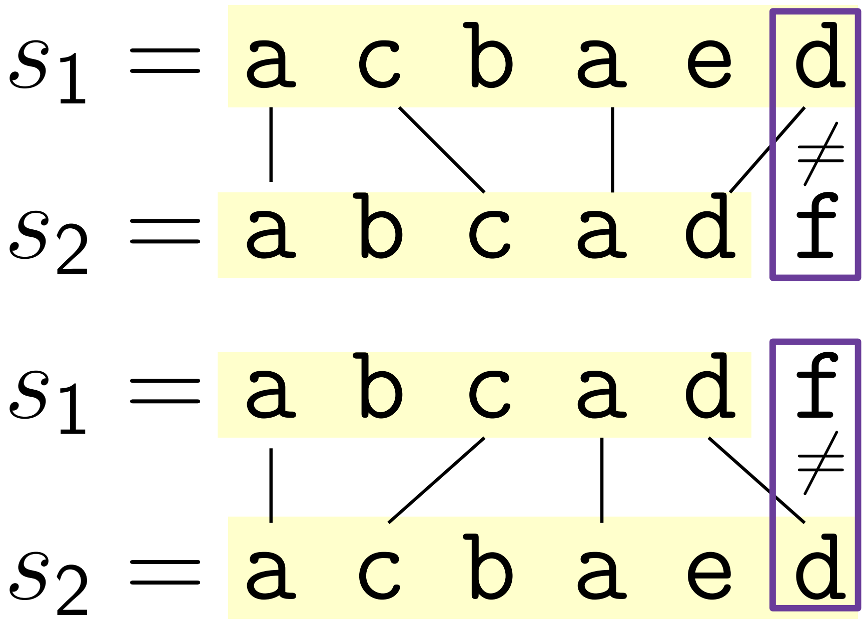
Filling the Table

Approach: figure out $L(n,m)$, and this will give us the general form of $L(i,j)$.

Let's make a recurrence :)

do we really need all of these?

- **Idea:** Recursively compute $L(n-1,j)$ for each $j \leq m$, and $L(i,m-1)$ for each $i \leq n$.
- If the last character of s_1 doesn't match the last of s_2, i.e., $s_1[n] \neq s_2[m]$, then . . .
  - $L(n,m) = \begin{cases} \max\{\, L(n-1,m), \quad L(n,m-1) \,\} & \text{, if } s_1[n] \neq s_2[m] \\ ?-----------------? & ,?-----------? \end{cases}$



$L(n,m-1)$

$L(n-1,m)$

Do we need to consider something else?

No :)

Filling the Table (part 2)

Note: we only really need $L(n, m)$, and this will give us the general form of $L(i, j)$.

Let's make a recurrence :)

- **Idea:** Recursively compute $L(n-1, m)$, $L(n, m-1)$, $L(n-1, m-1)$
  - $L(n, m) = \begin{cases} \max\{L(n, m-1), L(n-1, m)\} & , \text{ if } s_1[n] \neq s_2[m] \\ 1 + L(n-1, m-1) & , \text{ if } s_1[n] = s_2[m] \end{cases}$

$s_1 =$ a c b a e d

$s_2 =$ a b c a d

$L(n-1, m-1)$

- **Base Cases:**
  - $L(0, j) = 0$ for every $j \in \{1, ..., m\}$
  - $L(i, 0) = 0$ for every $i \in \{1, ..., n\}$

Why is this enough?

- $s_1[n]$ could match to an earlier $s_2[j]$, then $L(n, m) = 1 + L(n-1, j-1)$
- **But:** $L(n-1, m-1) \geq L(n-1, j-1)$, and since $s_1[n] = s_2[j] = s_2[m]$, **replace** the $(s_1[n], s_2[j])$ match with $(s_1[n], s_2[m])$

Filling the Table: an example

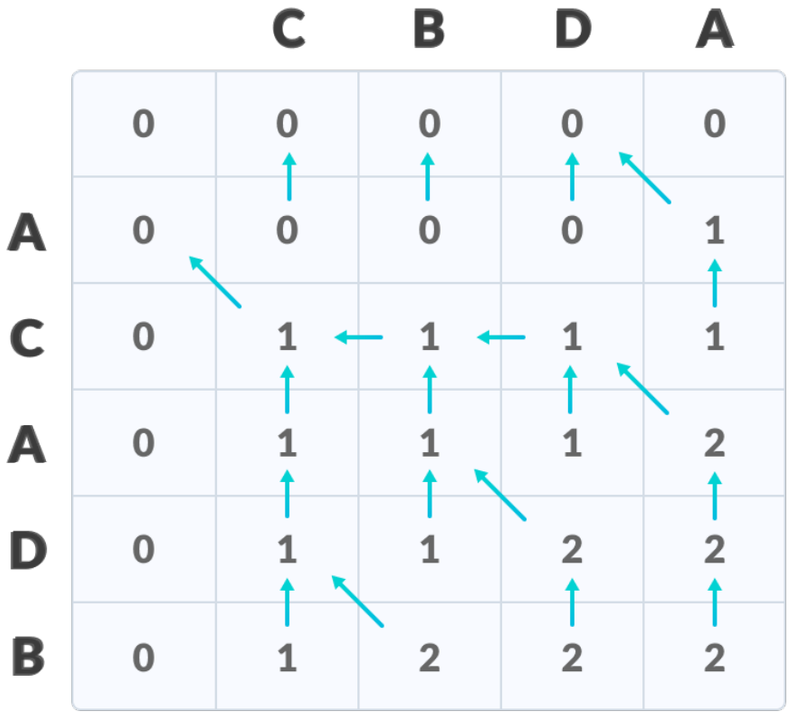
$L(0, j) = 0$ for every $j \in \{1, ..., m\}$

$L(i, 0) = 0$ for every $i \in \{1, ..., n\}$

$$L(i,j) = \begin{cases} \max\{L(i, j-1), L(i-1, j)\} & , \text{ if } s_1[i] \neq s_2[j] \\ 1 + L(i-1, j-1) & , \text{ if } s_1[i] = s_2[j] \end{cases}$$

- Fill in the first row and column as in the base case.
- Fill the first row as in our recurrence.
- The arrows show where the value came from
- Fill in the rest following the recurrence
- bottom right is the **length** of an LCS.

How do we find a longest common subsequence?

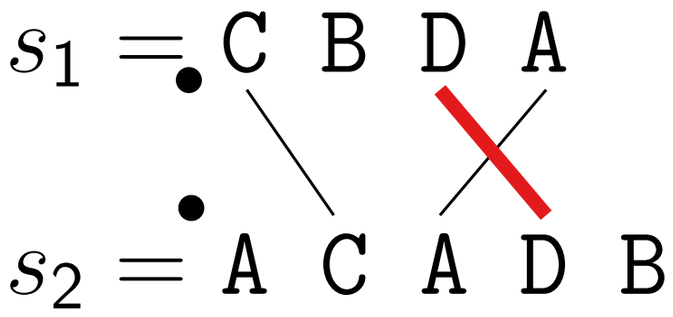


Backtracking the table

- Recover an LCS by "tracing" the table **backwards**, i.e., starting bottom-right.
- Each move: $\begin{cases} \text{up or left}, & \text{if } s_1[i] \neq s_2[j] \\ \text{diagonally up \& left}, & \text{if } s_1[i] = s_2[j] \end{cases}$

How does each move translate to subsequences?

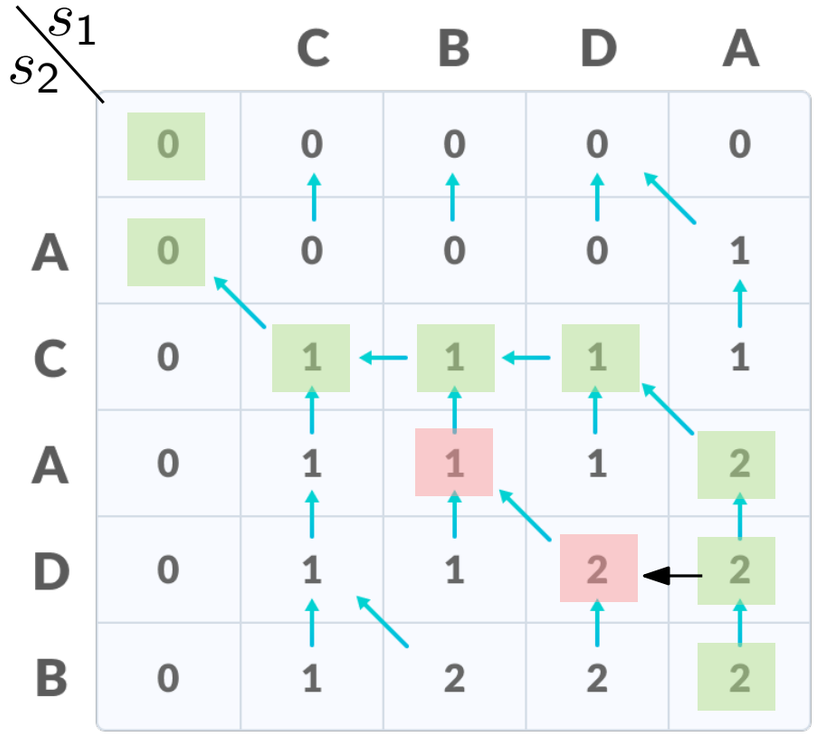
- up := skip in s_2
- left := skip in s_1
- diagonal := use this character



Output: by index

s_1: 1, 4
s_2: 2, 3

s_1: 1, 3
s_2: 2, 4



Note: **due to ties**, when moving "up or left" both options can be valid. These ties lead to **different optimal solutions**.

Runtime Analysis

$L(0, j) = 0$ for every $j \in \{1, ..., m\}$ — $O(m)$

$L(i, 0) = 0$ for every $i \in \{1, ..., n\}$ — $O(n)$

$$L(i,j) = \begin{cases} \max\{L(i, j-1), L(i-1, j)\} & , \text{ if } s_1[i] \neq s_2[j] \\ 1 + L(i-1, j-1) & , \text{ if } s_1[i] = s_2[j] \end{cases}$$

$O(1)$ per iteration

$O(nm)$ iterations

$O(nm)$

What would the implementation look like?

- iterate $i = 1...n$, and $j = 1...m$, nested loops

Backtracking?

$O(n + m)$

- Each move: $\begin{cases} \text{up or left,} & \text{if } s_1[i] \neq s_2[j] \\ \text{diagonally up \& left,} & \text{if } s_1[i] = s_2[j] \end{cases}$
  - up := skip in s_2
  - left := skip in s_1
  - diagonal := use this character

Total: $O(nm)$

LCS: Summary [GT 12.5 , CLRS 15.4]

Input: Two strings s_1 and s_2 where $|s_1| = n$ and $|s_2| = m$.
Goal: Compute a longest common subsequnce (LCS) of s_1 and s_2

By Dynamic Programming, we gave an algorithm that:

- returns an LCS in $O(nm)$ time using
- a **table** with $O(nm)$ entries, each of $O(1)$ size: $O(nm)$ total size.

Exercises:

- Implement the algorithm described here, both in the recursive memoization version and the bottom-up iterative version.
- To avoid backtracking, it is possible to maintain a solution for each entry in the table: in addition to the value L(i, j), store an LCS of $s_1[1 \dots i]$ and $s_2[1 \dots j]$.
  (i) What is the new runtime? (ii) What would the new size of the table be?

Roadmap for Dynamic Programming

1. **Recursive Structure:** How problem can be decomposed into simple subproblems and how global optimal solution relates to optimal solutions to these subproblems.
2. **Table:** Define an array indexed by the parameters that define subproblems, to store the optimal value for each subproblem (make sure one of the **sub**problems actually equals the whole problem).
3. **Recurrence:** Based on the recursive structure of the problem, describe a recurrence relation to compute the table values (including degenerate or base cases).
4. **Bottom-up:** Write an iterative algorithm to compute values in the array, in a bottom-up fashion, following the recurrence.
5. **Backtracking:** Use computed array values to find a solution achieving the best value (can require storing additional information about choices made while filling in the table).

Problems to Study

Longest Common Subsequence (LCS) [GT 12.5 , CLRS 15.4]

- Measuring the similarity of two strings, e.g., two strands of DNA.

Scheduling Telescope Time [GT 12.3]

- Activity Selection, where activities have **benefits**

Knapsack returns [GT12.6], [CLRS 16.2 (+exercise 35-7)]

- Can we really solve it in polynomial time?

Travelling Salesperson (TSP)

- Finding the fastest way to deliver all packages (visit all important locations) and return home.
- Exponential running times

Problem: Scheduling Telescope Time [GT §12.3]

- One important resource (eg, hubble telescope). Many users (scientists) want to use it.
- Each one needs it for a **specific time period** and has **expected benefit**.

Input: n time slots $(s_1, f_1), \ldots, (s_n, f_n)$. For each i, $s_i :=$ start time, $f_i :=$ finish time, $s_i < f_i$.
Each time slot has a positive numeric **benefit** b_i.
Objective: Plan a set of non-conflicting time slots to **maximize** the **benefit**.

What is a schedule/plan? Function p : events $\rightarrow$ {yes,no} (schedule or not).

When is a schedule **feasible**? No **conflicting** events are set to "yes".
$(p(T_i) = p(T_j) =$ yes$) \Rightarrow$ either $f_i \leq s_j$ or $f_j \leq s_i$.

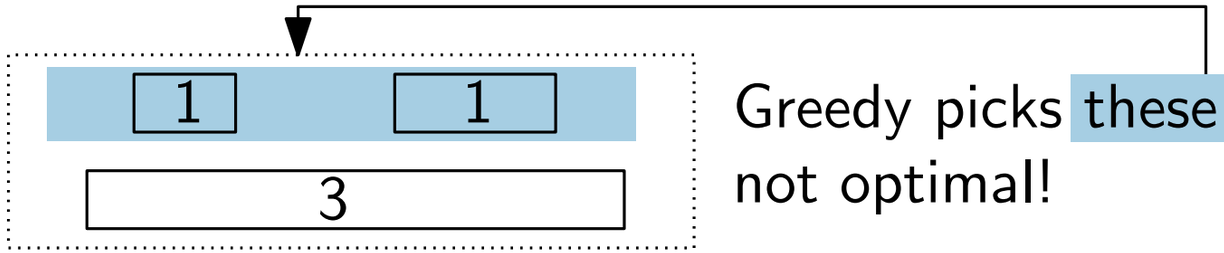
Looks like **Activity Scheduling** except . . . maximize **benefit** instead of the **number** of activities

Scheduling Telescope Time: Try Greedy first

Input: n time slots $(s_1, f_1), \ldots, (s_n, f_n)$. For each i, $s_i :=$ start time, $f_i :=$ finish time, $s_i < f_i$.
Each time slot has a positive numeric **benefit** b_i.
Objective: Plan a set of non-conflicting time slots to **maximize** the **benefit**.

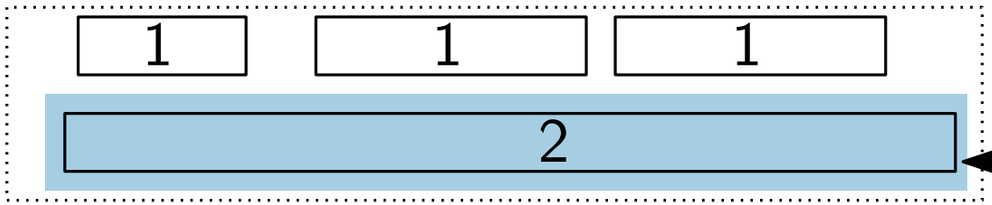
Try some greedy strategies:

- Greedy by finish time.



Greedy picks these two, not optimal!

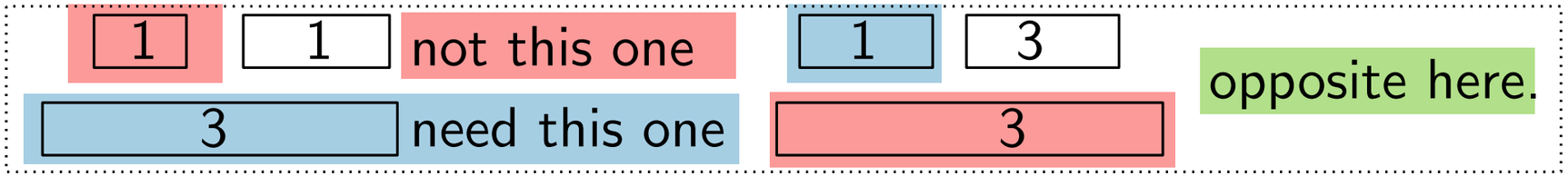
- Greedy by max benefit.



Greedy picks this one, not optimal!

- Greedy by max benefit/length. Top example counters optimality.

- Combinations (e.g., try by finish time then by max profit):



Idea: Need to maintain multiple candidate solutions. Which ones?

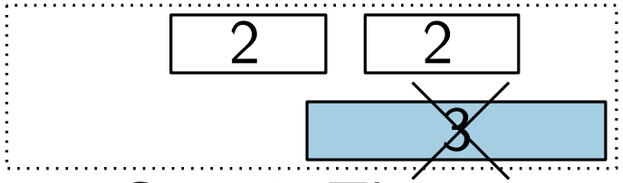
Scheduling Telescope Time: Subproblem Structure

Input: n time slots $(s_1, f_1), \ldots, (s_n, f_n)$. For each i, $s_i :=$ start time, $f_i :=$ finish time, $s_i < f_i$.
Each time slot has a positive numeric **benefit** b_i.
Objective: Plan a set of non-conflicting time slots to **maximize** the **benefit**.

Let's keep the ordering by finish time: $f_1 \leq f_2 \leq \cdots \leq f_n$.

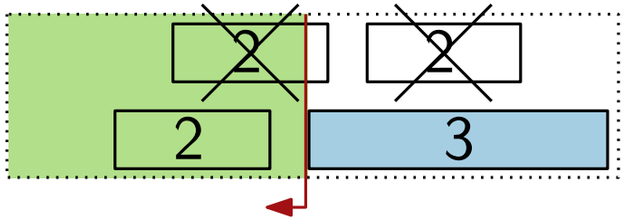
Recursive Thinking: Consider the last time slot (s_n, f_n).

- Case 1: No optimal solution uses (s_n, f_n).



Optimal solution := optimal solution for $(s_1, f_1), \ldots, (s_{n-1}, f_{n-1})$.

- Case 2: There is an optimal solution using (s_n, f_n).



Optimal solution := $\{n\} \cup$ (optimal solution for $(s_1, f_1), \ldots, (s_k, f_k)$),
where $k :=$ largest index i such that $f_i \leq s_n$

$\rightsquigarrow$ recursive algorithm!

Scheduling Telescope Time: Max Benefit By Recursion

Input: n time slots $(s_1, f_1), \ldots, (s_n, f_n)$. For each i, $s_i :=$ start time, $f_i :=$ finish time, $s_i < f_i$.
Each time slot has a positive numeric **benefit** b_i.
Objective: Plan a set of non-conflicting time slots to **maximize** the **benefit**.

Notation for pseudocode: array `A` where `A[i].s` $:= s_i$, `A[i].f` $:= f_i$, and `A[i].b` $:= b_i$,

Before calling the recursive algo, sort `A` by finish time: `A[1].f` $\leq$ `A[2].f` $\leq \cdots \leq$ `A[n].f`

And compute the **previous index** array `p` where, for each time slot `i` $\in \{1, \ldots, n\}$,

`p[i]` := largest index k such that `A[k].f` $\leq$ `A[i].s`; and if no such index, set `p[i] = 0`

```
# Return the maximum benefit possible from activities A[1],...,A[i]
function RecOpt(int i):
    if i == 0:  return 0
    else:  return max( RecOpt(i-1), A[i].b + RecOpt(p[i]) )
```

Correctness: Clear from the previous slide.

Runtime? Const. c such that:

$$T(n) = T(n-1) + T(p[n]) + c \leq T(n-1) + T(n-2) + c$$

Fibonacci !? Exponential :(

$\rightsquigarrow O(n)$ time by memoization (or bottom-up iteration)

Scheduling Telescope Time: Dyanmic Programming

Input: n time slots $(s_1, f_1), \ldots, (s_n, f_n)$. For each i, $s_i :=$ start time, $f_i :=$ finish time, $s_i < f_i$.
Each time slot has a positive numeric **benefit** b_i.
Objective: Plan a set of non-conflicting time slots to **maximize** the **benefit**.

Setup: memo array `OPT` so that `OPT[i]:=null` for each `i` $\in \{1, \ldots, n\}$, and set `OPT[0]:=0`

```
function MemRecOpt(int i):
   if OPT[i] == null:
      OPT[i]:= max(MemRecOpt(i-1), A[i].b +MemRecOpt(p[i]))
   return OPT[i]
```

Exercise: bottom-up iteration

```
OPT[n]:=
  MemRecOpt(n)
```

```
function reportPlan(array OPT)
    plan = {}; i = length(OPT);
    while i > 0:
        if OPT[i] == OPT[i-1]:  i:= i-1 // skip i
        else:  // schedule i
            add A[i] to the plan; i:= p[i]
    return plan
```

What about the plan?

Walk from n to 1 through `OPT`

Runtime:
$O(n)$: `p[i]` precomputed, and `A` already sorted;
$O(n \log n)$ otherwise.

Exercise: Compute `p` from sorted `A` in $O(n \log n)$ time.

Scheduling Telescope Time: Summary [GT §12.3]

Input: n time slots $(s_1, f_1), \ldots, (s_n, f_n)$. For each i, $s_i :=$ start time, $f_i :=$ finish time, $s_i < f_i$.
Each time slot has a positive numeric **benefit** b_i.
Objective: Plan a set of non-conflicting time slots to **maximize** the **benefit**.

Main result: $O(n \log n)$ time dynamic program to find an optimal solution.

- **Key idea:** Time slots can be naturally ordered by finish time.
- **Recursive Structure:** To determine if time slot i is used, look earlier in the order: best solution without i, and the best solution compatible with i.
- **Table** `OPT` indexed by time slots in this order. The value stored in the table is the optimal benefit that can be obtained by the first i time slots.

Exercises:

- Write out the pseudocode for the bottom-up iteration version.
- Compute `p` from sorted `A` in $O(n \log n)$ time.

Problems to Study

Longest Common Subsequence (LCS) [GT 12.5 , CLRS 15.4]

- Measuring the similarity of two strings, e.g., two strands of DNA.

Scheduling Telescope Time [GT 12.3]

- Activity Selection, where activities have **benefits**

Knapsack returns [GT12.6], [CLRS 16.2 (+exercise 35-7)]

- Can we really solve it in polynomial time?

Travelling Salesperson (TSP)

- Finding the fastest way to deliver all packages (visit all important locations) and return home.
- Exponential running times

KNAPSACK [GT§12.6], [CLRS §16.2 (+exercise 35-7)]

Input: Given a weight capacity $W \geq 0$ and n objects with positive integer weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$ respectively, i.e., object i has weight w_i and value v_i.

Goal: Pick a subset S of the given objects of maximum value and total weight at most W.

Greedy approach we tried:

- Pick coins by *value density*: $\frac{v_i}{w_i}$ (not optimal)

Consider 3 coins where the (w_i, v_i) are (3,5), (2,3), (2,3), and capacity $W = 4$.

Greedy gets the first coin (value$= 5$), but then cannot take any more.
It is better to take the other two coins (value$= 6$).

We need a more powerful technique (Dynamic Programming $\rightarrow$ week 3). Let's go!
Or an easier problem, see FRACTIONAL KNAPSACK [GT §10.1]

Knapsack

Input: Given a weight capacity $W \geq 0$ and n objects with positive integer weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$ respectively, i.e., object i has weight w_i and value v_i.

Goal: Pick a subset S of the given objects of maximum value and total weight at most W.

Roadmap:

1. **Recursive Structure:**
2. **Table:**
3. **Recurrence:**
4. **Bottom-up:**
5. **Backtracking:**

KNAPSACK: Table and Recurrence

Input: Given a weight capacity $W \geq 0$ and n objects with positive integer weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$ respectively, i.e., object i has weight w_i and value v_i.

Goal: Pick a subset S of the given objects of maximum value and total weight at most W.

Table: `OPT[j,Z]` := is the maximum value of a subset S of the first `j` objects where the total weight of S is at most `Z`.

How big is the table? `j` $\in \{1, \ldots, n\}$, and `Z` $\in \{0, 1, \ldots, W\}$

$\rightsquigarrow O(nW)$ entries

Recurrence: but, what if $w_j >$ `Z` ?

`OPT[j,Z] := max(OPT[j-1,Z],` v_j `+ OPT[j-1, Z-`w_j`]`

KNAPSACK: Table and Recurrence

Input: Given a weight capacity $W \geq 0$ and n objects with positive integer weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$ respectively, i.e., object i has weight w_i and value v_i.

Goal: Pick a subset S of the given objects of maximum value and total weight at most W.

Table: `OPT[j,Z]` := is the maximum value of a subset S of the first `j` objects where the total weight of S is at most `Z`.

How big is the table? `j` $\in \{1, \ldots, n\}$, and `Z` $\in \{0, 1, \ldots, W\}$

$\rightsquigarrow O(nW)$ entries

Recurrence: Base case? if $j = 0$, `OPT[j,Z] := 0`

$$\texttt{OPT[j,Z]} := \begin{cases} \texttt{max(OPT[j-1,Z],}\ v_j\ \texttt{+ OPT[j-1,}\ \texttt{Z-}w_j\texttt{]} & \text{if } w_j \leq \texttt{Z} \\ \texttt{OPT[j-1,Z]} & \text{otherwise } (w_j > \texttt{Z}) \end{cases}$$

KNAPSACK: Bottom-up Iteration

Input: Given a weight capacity $W \geq 0$ and n objects with positive integer weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$ respectively, i.e., object i has weight w_i and value v_i.

Goal: Pick a subset S of the given objects of maximum value and total weight at most W.

Table: `OPT[j,Z]` := is the maximum value of a subset S of the first `j` objects where the total weight of S is at most `Z`.

Recurrence:

$$\texttt{OPT[j,Z]} := \begin{cases} 0 \text{ if } j = 0 \text{ (base case)} \\ \texttt{max(OPT[j-1,Z],}\ v_j + \texttt{OPT[j-1,}\ \texttt{Z-}w_j\texttt{])} \text{ if } w_j \leq \texttt{Z} \\ \texttt{OPT[j-1,Z]} \quad \text{otherwise } (w_j > \texttt{Z}) \end{cases}$$

Bottom-up Algo:

```
for each Z from 0 to W, set OPT[0,Z]:= 0 // base case        O(W)
for Z from 0 to W, for j from 1 to n   O(nW) iterations , O(1) time each
.   if w_j > Z        then OPT[j,Z]:=OPT[j-1,Z]                    O(1)
.   else (w_j ≤ Z)   OPT[j,Z]:=max(OPT[j-1,Z], v_j +OPT[j-1,Z-w_j)  O(1)
end for end for
```

Runtime: $O(nW)$ time in total.

KNAPSACK: Reporting Optimal Solution

Input: Given a weight capacity $W \geq 0$ and n objects with positive integer weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$ respectively, i.e., object i has weight w_i and value v_i.

Goal: Pick a subset S of the given objects of maximum value and total weight at most W.

Table: `OPT[j,Z]` := is the maximum value of a subset S of the first `j` objects where the total weight of S is at most `Z`.

Main Idea: Step backwards through the objects and either

- report object `j` if ... `OPT[j, Z]` == v_j+`OPT[j-1, Z-`w_j`]` and revise `j:=j-1`, `Z:=Z-`w_j
- skip the object and revise `j:= j-1`

both cases have `j:=j-1`

```
objs := null; // objs :=picked objects
for j from n to 1
    .if OPT[j,Z]==v_j+OPT[j-1, Z-w_j] then
    .    objs.add(j), Z:=Z-w_j
end for
```

Runtime:
$O(n)$

KNAPSACK: Summary

Input: Given a weight capacity $W \geq 0$ and n objects with positive integer weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$ respectively, i.e., object i has weight w_i and value v_i.

Goal: Pick a subset S of the given objects of maximum value and total weight at most W.

Main result: $O(nW)$ time dynamic program to find an optimal solution.

- **Key idea:** Indexing the table by weight limit W.
- **Recursive Structure:** To determine if object j should be used, compare dropping it (`OPT[j-1,Z]`) with keeping it and taking the best of the first `j-1` objects putting aside weight for `j` (v_j+`OPT[j-1,Z-`w_j`]`)

Remarks on runtime:

This is not polynomial time! The size of the input W is actually $\log W$

eg, $(\sum_{i=1}^{n} w_i)$, $W \in \Theta(2^n) \Rightarrow$ input size is $\Theta(n^2)$, but runtime is $O(n2^n)$

But this **kind of** the best we can do, since the problem is **NP-complete** [see next week!]

This is sometimes called **0-1 Knapsack**: 0 or 1 of each object. What if we allow **repetition**?

KNAPSACK: With repetition

Input: Given a weight capacity $W \geq 0$ and n objects with positive integer weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$ respectively, i.e., object i has weight w_i and value v_i.
Goal: Pick a multiset S (with repetition allowed) of the given objects of maximum value and total weight at most W.

Table: `OPT[Z]` := is the maximum value of a multiset S picked from all objects where the weight of S is at most `Z`

Recurrence:

$$\texttt{OPT[Z]} := \begin{cases} 0 & \text{if } \texttt{Z} < w_j \text{ for every } j \in \{1, \ldots, n\} \text{ (base case)} \\ \texttt{max(\{}v_j\texttt{+OPT[Z-}w_j\texttt{]}\texttt{ : } j \in \{1, \ldots, n\} \texttt{ \}} & \text{otherwise} \\ \quad \text{and } w_j \leq \texttt{Z} & \end{cases}$$

Table Size: $O(W)$. Computing the next entry: $O(n)$

Runtime: $O(nW)$.

Bottom-up and backtracking? Exercise :)

For more exercises related to KNAPSACK, try the COIN CHANGING problem on the exercises slide.

Problems to Study

Longest Common Subsequence (LCS) [GT 12.5 , CLRS 15.4]

- Measuring the similarity of two strings, e.g., two strands of DNA.

Scheduling Telescope Time [GT 12.3]

- Activity Selection, where activities have **benefits**

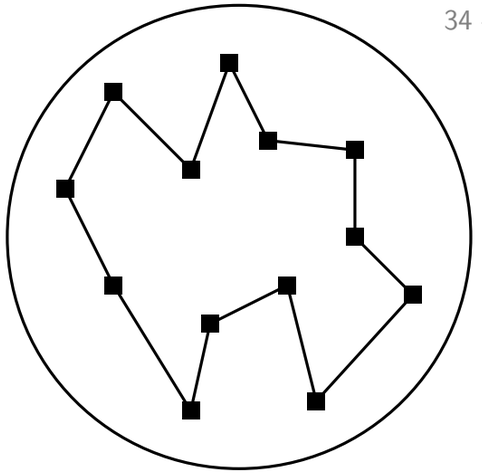
Knapsack returns [GT12.6], [CLRS 16.2 (+exercise 35-7)]

- Pseudo polynomial time algorithm.

Travelling Salesperson (TSP)

- Finding the fastest way to deliver all packages (visit all important locations) and return home.
- Exponential running times

Traveling Salesperson Problem (TSP)



Input Complete directed graph $G = (V, E)$ with n vertices and edge weights $c\colon E \to \mathbb{Q}_{\geq 0}$

Output A Hamiltonian cycle $C = (v_1, \ldots, v_n, v_{n+1} = v_1)$ of G, of minimum weight $\sum_{i=1}^{n} c(v_i, v_{i+1})$.

cycle in G using all vertices

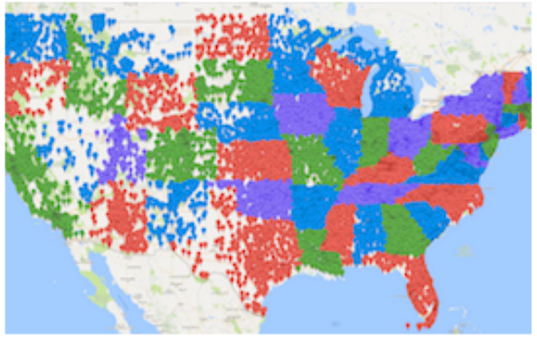
Brute-Force? Brutally slow ... improve to a better (exponential) algorithm

- Each tour is a permutation of the vertices.
- Pick a permutation with the smallest weight.

Runtime: $\Theta(n! \cdot n) \simeq n \cdot 2^{\Theta(n \log n)}$

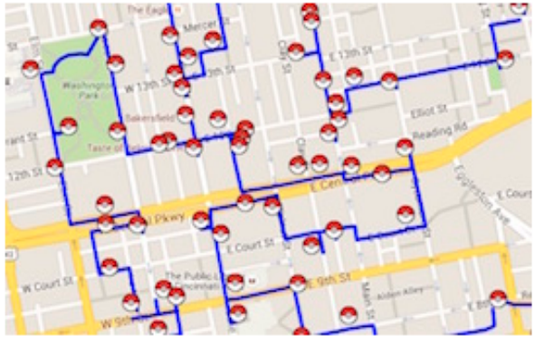
Other Road Trips https://www.math.uwaterloo.ca/tsp

49,687 pubs the UK.



49,603 US historic places.

24,727 pubs in the UK.



How to catch 'em all.

Motivation: exact exponential algorithms

- can be "fast" for **medium-sized** instances

$\rightsquigarrow$ e.g.: $n^4 > 1.2^n$ for $n \leq 100$

$\rightsquigarrow$ e.g.: using a **clever** algorithm TSP solvable exactly for $n \leq 2000$ and specialized instances with $n \leq 85900$
(even better results shown here: https://www.math.uwaterloo.ca/tsp)

$\rightsquigarrow$ "hidden" constants in polynomial time algorithms: $2^{100} \cdot n > 2^n$ for $n \leq 100$

- theoretical interest

Bellman-Held-Karp-Algorithm

Technique: Dynamic Programming!

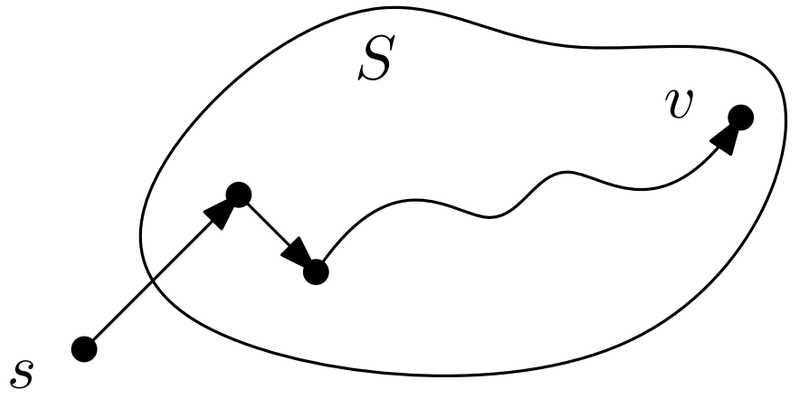
Recursive structure and Table

Select any starting vertex $s \in V$.

all elements of V except s

For each $S \subseteq V - s$ and $v \in S$, let:

$\text{OPT}[S, v] =$ length of the shortest s-v-path that visits precisely the vertices of $S \cup \{s\}$.



Richard M. Karp

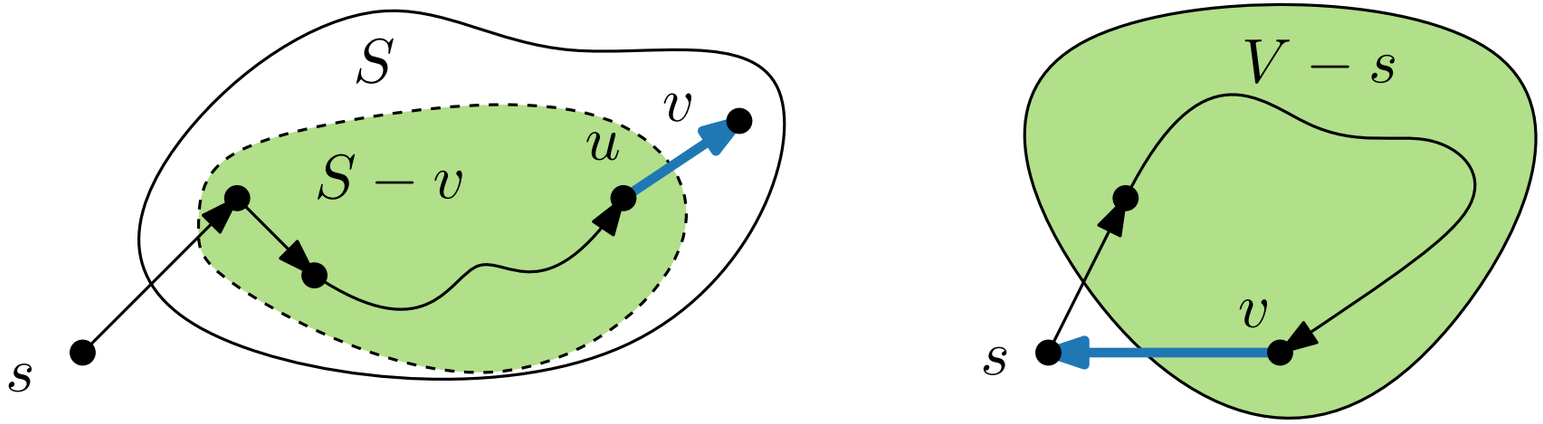
Richard E. Bellman

Bellman-Held-Karp-Algorithm (recurrence)

The base case: $S = \{v\}$, is easy: $\mathrm{OPT}[\{v\}, v] = c(s, v)$.

When $|S| \geq 2$, we compute $\mathrm{OPT}[S, v]$ recursively:

$$\mathrm{OPT}[S, v] = \min\{\ \mathrm{OPT}[S - v, u] + c(u, v)\ \mid u \in S - v\ \}$$



After computing $\mathrm{OPT}[S, v]$ for each $S \subseteq V - s$, the optimal solution is obtained as follows:

$$\mathrm{OPT} = \min\{\ \mathrm{OPT}[V - s, v] + c(v, s)\ \mid v \in V - s\ \}$$

Bottom-up Algorithm

Algorithm Bellmann-Held-Karp(G, c)

foreach $v \in V - s$ **do**
  OPT$[\{v\}, v] = c(s, v)$

for $j = 2$ **to** $n - 1$ **do** } $O(2^n)$
  foreach $S \subseteq V - s$ with $|S| = j$ **do** }
    foreach $v \in S$ **do** } $O(n)$
      OPT$[S, v] :=$ min$\{$ OPT$[S - v, u] + c(u, v) \mid u \in S - v \}$

return min$\{$ OPT$[V - s, v] + c(v, s) \mid v \in V - s \}$

Runtime: min operation (innermost loop) is called $O(2^n \cdot n)$ times
Each min op: $O(n)$ time.
Thus, $O(2^n \cdot n^2)$ total.

Table size, i.e., Space (memory) usage: $\Theta(2^n \cdot n)$

Computing the table for size j only uses table-values for $j - 1$. **Exercise:** Can this be used to save memory? If so, how?

But, how can we use this to determine a shortest tour?

Backtracking the DP table!
Exercise: figure it out ;)

Compare: $O(2^n \cdot n^2)$ with $2^{O(n \log n)} \cdot n$ for Brute-Force

Exercises

Introduction: Fibonacci Numbers

1. `memoF` has $\Theta(n)$ entries $\sim$ memory. Can we compute F_n using $O(1)$ memory?

Longest Common Subsequence (LCS) [GT 12.5 , CLRS 15.4]

1. Implement the algorithm described here, both in the recursive memoization version and the bottom-up iterative version.
2. To avoid backtracking, it is possible to maintain a solution for each entry in the table: in addition to the value L(i, j), store an LCS of $s_1[1 \ldots i]$ and $s_2[1 \ldots j]$. (i) What is the new runtime? (ii) What would the new size of the table be?

Scheduling Telescope Time [GT 12.3]

1. Write out the pseudocode for the bottom-up iteration version, and analyze the runtime.
2. Compute `p` from sorted `A` in $O(n \log n)$ time.

Knapsack [GT12.6], [CLRS 16.2 (+exercise 35-7)]

1. (not from the slides) **Coin Changing:** Given a supply of coins of denominations `d[1] < d[2] < ... < d[n]`, we wish to make change for a value v; that is, to find a set of coins whose total value is v. This might not be possible: eg, if the denominations are 5 and 10 then we can make change for 15 but not for 12. Give $O(nv)$ time dynamic-programming algorithms for the following problems.
   (a) Each coin denomination may be used an **unlimited number of times**.
   If it is possible to make change for v, output the coins used. Otherwise: output $\textsc{No}$.
   (b) Each coin denomination may be used **at most once**.
   If it is possible to make change for v, output the coins used. Otherwise: output $\textsc{No}$.
   (c) Repeat (a) and (b), but now if it is possible to make change, the output should be the fewest coins possible to do so.

Travelling Salesperson (TSP)

1. Provide an algorithm to backtrack the table.
2. Similar to the LCS case, to compute the table for sets of size j (current column), only need the entries for sets of size $j - 1$ (previous column). This reduces the saved table entries, but prevents easy backtracking. So we should store more **info** in the table.
   (i) What is the **info** to be added? (ii) What is the new runtime? (iii) What is the size of the largest such "column" j ?

Summary (Weeks 1–3), and Next Week

Standard Algorithmic Paradigms:

- Greedy: Grow **one** solution step-by-step according to a **clever rule**
- Divide and Conquer: **Split cleverly** into pieces, solve the pieces, **combine cleverly**.
- Dynamic Programming:
  Optimal solution could arise **several different ways**, try them all (recursively).

Next Week: Classifying computational problems.

or, how can we distinguish “easy” (fast to solve) from “hard” (slow to solve)